

Smaller tensors. Every bit preserved.

Compress the structure in BF16 exponents, then decode where training needs the values.

1.45–1.52×

Ideal matched-model ratio*

01 Separate the bits

BF16 input: 16 bits per value

SIGN EXPONENT FRACTION



Exponent byte → encode

Concentrated distributions carry fewer bits.

Sign + fraction → keep

One unchanged byte per value.

Bitwise split; no quantization or rounding.

02 Code the exponents

Short codes for common symbols

Estimate center from ≤ 4096 values; reuse a codebook.



Common symbols

Short Huffman codes

Direct prefixes: ≤ 8 bits

Rare symbols

Escape prefix + 8-bit symbol literal

At most 16 bits; every exponent remains representable.

Search tail cutoffs; optimize bounded code lengths.

03 Pack for the GPU

Fixed payloads; independent streams

256 streams per block

Cyclic assignment; interleaved payload words



Stream full? Store the raw tail.

Remaining exponent bytes + stream metadata

Unchanged sign–fraction bytes

Shared GPU arena allocates and reuses overflow storage.

BUILT FOR REPEATED TRAINING UPDATES

Keep state compressed between uses

Weights + momentum

Decode + update

Encode new state

Fused pointwise updates avoid dense input intermediates. Gradients stay dense.

Saved activations decode for backward; matrix products reconstruct before dense GEMM.

LOSSLESS ROUND TRIP

Recover the original bits

Decode exponents + rejoin the side byte

$$\text{decode}(\text{encode}(u)) = u$$

All 16 bits, including signed zeros, subnormals, infinities, and NaN payloads.

Storage is exact; arithmetic still uses BF16 rounding.

*Ideal ratio $16/[8 + H(E)]$ across the paper's four theoretical distributions. Actual storage also includes fixed stream budgets, overflow, padding, and metadata.